

Sérialisation

Jonathan Lejeune

Sorbonne Université/LIP6

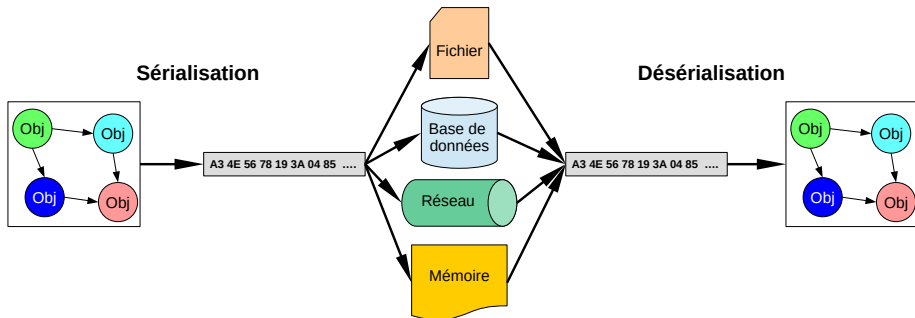
SRCS – Master 1 SAR 2023/2024

sources :

Développons en Java, Jean-Michel Doudoux

Définitions

- **Sérialisation** : Transformer la valeur d'un objet en un flux d'octets
synonymes : marshalling, linéariser
- **Désérialisation** : Transformer un flux d'octets pour construire un objet
synonymes : unmarshalling, délinéariser



A quoi ça sert ?

- Rendre persistants des objets
- Stocker des objets momentanément inutilisés
- Sauvegarder une configuration
- Échanger des objets
- Créer une copie intégrale d'un objet (clonage en profondeur)

Comment faire ?

- Manuellement
- Automatiquement via l'environnement d'exécution
ex : sérialisation JVM
- Via un outil ou une API
ex : Protobuf, Avro, Parquet, XDR

Binaire

100011010101011010101101011010101010011100011010

- ✓ Moins volumineux
- ✓ Plus rapide à encoder/décoder
- ✗ Modification directe difficile

Textuel

```
<?xml version="1.0" encoding="UTF-8"?>  
  <parent>
```

....

- ✓ Portabilité $\Rightarrow$ utilisation d'une structuration standard (XML, JSON)
- ✓ Modification directe facile
- ✗ Plus volumineux
- ✗ Coûteux en ressources pour être traité

- Comment assurer le fait qu'un objet sérialisé soit toujours compatible à sa désérialisation ?
- Comment assurer un format portable ?
- Comment gérer et maintenir les références ?
- Comment gérer les cycles ?
- Comment spécifier les parties du graphe que l'on ne souhaite pas sérialiser ?

Caractéristiques

- Fonctionnalité offerte par la JVM
- Enrichissement de l'API des I/O
- Un format portable pour sauvegarder/charger un objet indépendamment de l'OS

Outils offerts au programmeur

- Mot-clé `transient`
- Interfaces : `Serializable`, `Externalizable`
- Classes : `ObjectInputStream`, `ObjectOutputStream`

L'interface `Serializable`

- Interface marqueur du package `java.io`
⇒ n'offre pas de méthodes
- Mécanisme de sérialisation binaire par défaut
- Interface qui doit être obligatoirement héritée par :
 - soit la classe de l'objet que l'on veut sérialiser
 - soit une de ses classes mères
 - soit une de ses interfaces mères

sinon `java.io.NotSerializableException`

Le mot clé `transient`

Permet de marquer un attribut pour ne pas le prendre en compte dans le processus de sérialisation.

Sérialiser un graphe d'objets

Principe

Pour pouvoir sérialiser un graphe d'objets, toute référence doit désigner un objet qui est de type `Serializable`

Que faire si la classe d'un attribut n'est pas `Serializable` ?

- Si on est éditeur de la classe
⇒ la faire étendre de `Serializable`
- Si on n'est pas éditeur de la classe et classe non final
⇒ créer une classe fille `Serializable` et utiliser cette sous-classe
- Si on souhaite ignorer l'attribut
⇒ ajouter le mot clé `transient`

Quelques classes sérialisable et non sérialisable de l'API java

Classes non sérialisables

- Object
- Classes singletons : Math, System
- Classes de contexte d'exécution : Runtime, Thread, Process, ClassLoader
- Classes de communication : InputStream, OutputStream, Socket

Classe sérialisables

- Classe type de base : Integer, Long, Double, String, Boolean, ...
- Les classes implantant Throwable : Exception, Error
- la classe Class
- Toute classe d'implémentation concrète de collection
⇒ les éléments de la collection doivent être aussi Serializable

La classe `ObjectOutputStream`

Caractéristiques

- Décorateur de `OutputStream`
- Offre toutes les méthodes de `DataOutputStream`
- Sériailise un objet ou un graphe d'objets :
 - Parcours des objets un à un
 - Tenir compte des références d'objets déjà sérialisés

La méthode `void writeObject(Object o)`

- Sériailise le graphe d'objets dont l'objet racine est fourni en paramètre.
- Par défaut le mécanisme de sérialisation écrit pour chaque objet :
 - la classe de l'objet
 - les valeurs de champs
- Par défaut tous les champs d'un objet sont sérialisés sauf :
 - les attributs `static`
 - les attributs `transient`

La classe `ObjectInputStream`

Caractéristiques

- Décorateur de `InputStream`
- Offre toutes les méthodes de `DataInputStream`
- Déséréalise un objet ou un graphe d'objets à partir d'un flux

La méthode `Object readObject()`

- Renvoie une instance à caster vers le type présumé (faire éventuellement un `instanceof`)
- **La classe présumée doit pouvoir être chargée**
⇒ `ClassNotFoundException` sinon
- les champs `transient` sont initialisés avec la valeur `null`.
⇒ cet état doit être géré par l'objet pour éviter les `NullPointerException`
- **Attention : le constructeur de la classe n'est pas invoqué**

Objectif

Détecter la compatibilité ou l'incompatibilité entre la version actuelle d'une classe et la version renseignée dans des objets déjà sérialisés.

Comment faire ?

- Par convention, tout objet `Serializable` contient un attribut constant et statique de numéro de version

```
private static final long serialVersionUID = 1L;
```
- Si pas spécifié $\Rightarrow$ Warning à la compilation et attribution automatique par la JVM du numéro de version (peut être source de bug)
- Lors de la désérialisation, si le numéro de version des données ne correspond pas au numéro de la classe $\Rightarrow$ `InvalidClassException`

Compatibilité automatique des versions (1/2)

Principe

La sérialisation par défaut peut gérer automatiquement plusieurs évolutions d'une classe sans empêcher la désérialisation de données d'une version précédente

Précondition : le `serialVersionUID` doit rester identique

Cas d'incompatibilités

- Suppression de champs
- Échanger la hiérarchie de deux classes dans la hiérarchie
- Ajout d'un modificateur `static` ou `transient` à un champ
- Changer le type primitif d'un attribut
- Supprimer ou remplacer `Serializable` par `Externizable` et vice-versa
- Transformer une classe en une énumération et vice-versa

Cas de compatibilité

- Ajouter des champs : un champ non lu est initialisé par défaut
- Changer le modificateur d'accès d'un champ
- Retirer le modificateur `static` ou `transient` d'un champ
- Implanter l'interface `Serializable`
- Ajouter un type dans la hiérarchie des classes mères
- Supprimer un type dans la hiérarchie des classes mères

Cas où il existe une classe/interface mère Serializable

- Si on veut une classe fille Serializable $\Rightarrow \emptyset$
- Si on ne veut pas que la classe fille soit Serializable
personnaliser la sérialisation pour cette classe en définissant les méthodes `writeObject()` et `readObject()`

Cas où il n'existe pas de classe/interface mère Serializable

- Si on veut une classe fille Serializable
 $\Rightarrow$ implanter l'interface Serializable
- L'état des classes mère n'est pas pris en compte dans la sérialisation/désérialisation même si les champs sont accessibles dans la classe fille
- Les classes mères doivent avoir un constructeur par défaut
- Les champs de la classe mère doivent être gérés manuellement

Quand est-ce utile ?

Le mécanisme par défaut ne peut pas répondre à tous les besoins :

- contrôler plus finement les instances utilisées
- tenir compte des données sensibles (chiffrage)
- gérer la sérialisation de différentes versions de la classe
- améliorer les performances notamment avec des versions anciennes de Java

Mécanismes proposés par Java

Par finesse croissante :

- définir explicitement la liste des champs à inclure dans la sérialisation
- définir les méthodes `writeObject()` et `readObject()` (ou `writeReplace()` et `readResolve()`)
- implémenter l'interface `Externalizable`

Définition explicite des champs à sérialiser/désérialiser

- Champs statique de type `ObjectStreamField[]` et doit se nommer `serialPersistentFields`
- Un `ObjectStreamField` associe le nom de l'attribut `String` à son type `Class<>`
- **Attention** : annule l'effet de `transient`

```
public class Produit implements Serializable {  
  
    private String nom;  
    private Double prix;  
  
    private static final ObjectStreamField[] serialPersistentFields =  
        { new ObjectStreamField("nom", String.class),  
          new ObjectStreamField("prix", Double.class)};  
  
    //getter, setter  
    ...  
}
```

Les méthodes `writeObject()` et `readObject()`

Définition de deux méthodes qui seront invoqués en remplacement du mécanisme standard :

```
private void readObject(ObjectInputStream ois) throws IOException, ClassNotFoundException;  
private void writeObject(ObjectOutputStream oos) throws IOException;
```

```
public class Produit implements Serializable {  
  
    private String nom;  
    private Double prix;  
  
    private void readObject(ObjectInputStream ois) throws ... {  
        this.nom = (String) ois.readObject();  
        this.prix = (Double) ois.readObject();  
    }  
    private void writeObject(ObjectOutputStream oos) throws ... {  
        oos.writeObject(this.nom);  
        oos.writeObject(this.prix);  
    }  
    ...  
}
```

L'interface Externalizable

- Interface qui étend Serializable et qui offre deux méthodes :

```
void readExternal(ObjectInput in);  
void writeExternal(ObjectOutput out);
```

- Constructeur par défaut à définir obligatoirement dans les classes
- Annule et remplace le mécanisme readObject/writeObject
- Plus performant car on n'utilise pas l'introspection mais les méthodes de sérialisation sont publiques.

```
public class Produit  
    implements Externalizable {  
  
    private String nom;  
    private Double prix;  
  
    public Produit(){}  

```

```
    public void readExternal(ObjectInput in){  
        this.nom = (String) in.readUTF();  
        this.prix = (Double) in.readDouble();  
    }  
    public void writeExternal(ObjectOutput out) {  
        out.writeUTF(this.nom);  
        out.writeDouble(this.prix);  
    }  
    ...  
}
```

Les exceptions liées à la sérialisation

ObjectStreamException	Classe mère
InvalidClassException	– Incompatibilité dans les versions – Type d'un champ primitif ne correspond pas à la donnée sérialisée – Implantation de Externalizable sans constructeur par défaut – Implantation de Serializable mais classe mère non sérialisable n'a pas de constructeur par défaut
NotSerializableException	La classe n'est pas sérialisable
StreamCorruptedException	Flux corrompu ou invalide
WriteAbortedException	Une erreur est survenue durant l'écriture du flux
ClassNotFoundException	Impossible de charger la classe lors de la désérialisation

Exemple de Sérialisation

```
package srcs;

class Mere implements Serializable{
    private static final long serialVersionUID = 2L;
    private int j=-1;
}

class Foo extends Mere{
    private static final long serialVersionUID = 1L;
    private int i=4;
    private String s="ok";
    private Voisin v = new Voisin(this);
}

class Voisin implements Serializable{
    private static final long serialVersionUID = 3L;
    private int x = 0;
    private Foo voisin = null;
    public Voisin(Foo f) { this.voisin=f; }
}

ObjectOutputStream oos = new ObjectOutputStream(????)
oos.writeObject(new Foo());
```

Exemple de Sérialisation

